Ejercicio 1

Crea una clase llamada Rectangulo que reciba en su constructor primario dos atributos inmutables: ancho: Double y alto: Double.

Agrega una propiedad calculada llamada area de tipo Double que no almacene un valor en memoria, sino que use un get() de expresión única para devolver el resultado de multiplicar this.ancho * this.alto. Incluye una función main() que instancie la clase e imprima las dimensiones y el área calculada.

Ejercicio 2

Diseña una clase Estudiante que reciba nombreInicial: String y notaBase: Double en la cabecera (sin val ni var en los parámetros).

Define la propiedad nombre: String de forma que su get() personalizado devuelva el nombre en mayúsculas y sin espacios sobrantes (field.uppercase().trim()). El set() debe asignar el valor recibido directamente al campo de respaldo field. En main(), prueba cambiar el nombre y comprueba cómo se lee la propiedad.

Ejercicio 3

Crea una clase Termostato con una propiedad temperatura: Double inicializada en 20.0.

Sobrescribe su set() para que valide si la nueva temperatura está dentro del rango seguro (entre -10.0 y 45.0 grados). Si está dentro del rango, actualiza field; de lo contrario, muestra un mensaje de error y no modifiques field. En el main(), asigna una temperatura válida y otra inválida para ver el comportamiento.

Ejercicio 4

Crea una clase ContadorPasos con la propiedad pasos: Int inicializada en 0. La propiedad debe ser de lectura pública pero con modificación privada (private set).

Añade un método darPaso() que incremente pasos en 1 y un método reiniciar() que devuelva pasos a 0. En el main(), ejecuta ambos métodos e intenta asignar un valor a pasos directamente desde fuera para constatar que no compila.

Ejercicio 5

Diseña una clase VehiculoElectrico que reciba nivelBateria: Int en su constructor.

Añade una propiedad calculada llamada estadoBateria: String cuyo get() evalúe this.nivelBateria mediante una expresión when:

- Si es de 0 a 14 "Crítico"
- Si es de 15 a 49 "Bajo"
- Si es de 50 a 100 "Óptimo"

En el main(), modifica el porcentaje de batería y muestra cómo cambia el estado en tiempo real.

Ejercicio 6

Crea una clase ProductoTienda que reciba precioInicial: Double en el constructor.

Define la propiedad precio: Double. Su set(valor) debe rechazar precios negativos (conservando el valor previo), mientras que su get() debe añadir automáticamente un 10% por gastos de gestión sobre field cada vez que se consulte el precio. Pruébalo en main().

Ejercicio 7

Crea una clase UsuarioApp con una propiedad email: String y una propiedad emailVerificado: Boolean (esta última con private set).

El set() de email debe comprobar si el valor contiene un símbolo @. Si lo contiene, asigna el nuevo valor a field y cambia emailVerificado a true. Si no tiene @, muestra un error sin cambiar field. Usa un bloque init para inicializar el email usando el setter.

Ejercicio 8

Crea una clase TarjetaCredito que reciba limiteInicial: Double.

1. La propiedad deuda: Double debe tener private set e iniciar en 0.0.
2. La propiedad limite: Double se puede cambiar desde fuera, pero su set() debe rechazar cualquier límite que sea menor a la deuda actual.
3. Define una propiedad calculada creditoDisponible: Double (limite - deuda).
4. Crea un método realizarCompra(monto: Double) que añada a la deuda si hay crédito suficiente. Pruébalo en main().

Ejercicio 9

Crea una clase HabitacionHotel con numero: Int en su constructor primario.

- huesped: String: En su set(), elimina los espacios en blanco de los extremos y pone en mayúscula solo la primera letra del nombre. Si se intenta asignar un texto vacío o con espacios, debe fijar el valor "Sin Ocupar".
- diasEstancia: Int: En su set(), impide valores inferiores a 1.
- costoTotal: Double: Propiedad calculada que devuelva diasEstancia * 65.0.

Ejercicio 10

Diseña una clase BilleteraVirtual con las siguientes características:

- Propiedad saldo: Double inicializada en 0.0 con private set.
- Propiedad moneda: String (por defecto "EUR"). Su set() solo permite "EUR", "USD" o "GBP". Si recibe otra moneda, muestra un error y conserva la moneda original.
- Propiedad calculada esCuentaVIP: Boolean cuyo get() devuelva true si saldo > 10000.0 o si la moneda es "GBP".
- Métodos recargar(monto: Double) y retirar(monto: Double) con validación de saldo suficiente. Pruébalo en main().